ENCOMP Agenda

Backend · API REST

Teoria rápida → prática no projeto

3h30 · Node.js · Express · SQLite

O projeto de hoje

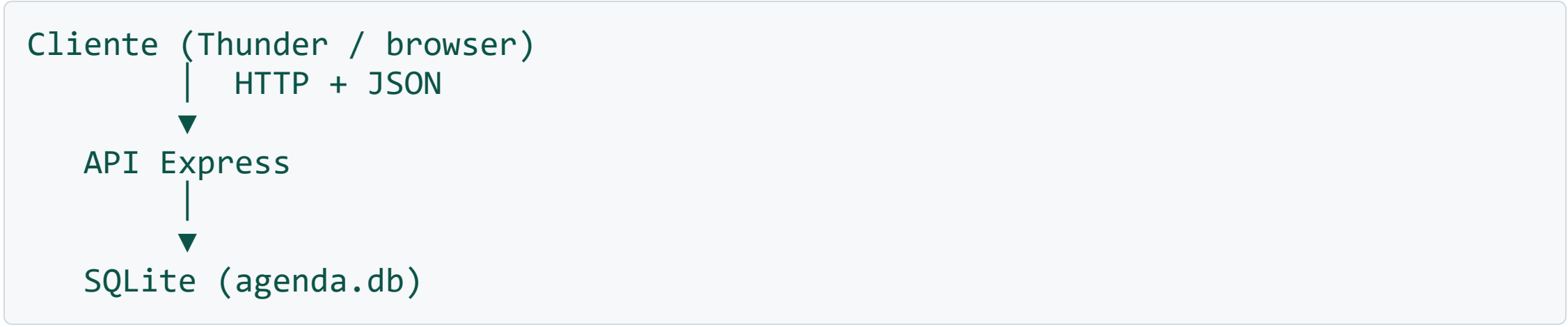
Vamos construir a **API da agenda de eventos**:

- CRUD completo em `/eventos`
- Persistência em **SQLite**
- Publicação no **Render** (instrutor)
- Amanhã o front consome essa URL

Front e back **separados**.

Teoria 1 · O que é uma API?

Contrato entre quem pede e quem responde.



O navegador **não** acessa o banco. Passa pela API.

Teoria 2 · HTTP + JSON

Parte	Exemplo no projeto
Método	`GET`, `POST`, `PUT`, `DELETE`
URL	`/eventos` ou `/eventos/:id`
Body	JSON no POST/PUT
Status	`200`, `201`, `204`, `400`, `404`

```
{ "titulo": "Minicurso", "data": "2026-10-23", "local": "Lab 2" }
```

Teoria 3 · REST = recurso + verbo

Recurso: **evento** → `/eventos`

Verbo	Caminho	Ação
GET	`/eventos`	listar
GET	`/eventos/:id`	um item
POST	`/eventos`	criar
PUT	`/eventos/:id`	atualizar
DELETE	`/eventos/:id`	remover

Isso é o **CRUD** que vamos codar.

Teoria 4 · Modelo do projeto

Campo	Obrigatório
`id`	gerado no servidor
`titulo`	sim
`data`	sim (`YYYY-MM-DD`)
`local`	sim
`status`	não (padrão `agendado`)
`descricao`	não

Também: `GET /health` para saber se a API está no ar.

Arquitetura do projeto (3 arquivos)

```
index.js            → sobe o servidor (/health)
db.js               → SQLite (tabela + seed)
routes/eventos.js   → CRUD de /eventos
```

Cada arquivo tem **uma responsabilidade**.

Mesma ideia de apps maiores, sem pastas demais.

Da teoria à prática

1. `index.js` + `GET /health`
2. `db.js` (banco)
3. `routes/eventos.js` → GET/POST
4. PUT + DELETE
5. Deploy (instrutor) + URL no quadro

Hora da prática

Abram a pasta `backend`. Digitem junto comigo.

Primeiro passo:

`index.js` com `GET /health` → `{ "ok": true }`

Vamos ao código.